

Developing and Evaluation a Efficient Automatic Differentiation Library for Convolutional Neural Networks in Julia

1st Jakub Wyłucki

Electrical Faculty

Warsaw University of Technology

Warsaw, Poland

01159628@pw.edu.pl

Abstract—I present an automatic differentiation (AD) library in Julia, tailored for optimizing convolutional neural networks (CNNs) using MNIST data. This library harnesses Julia’s computational efficiency, addressing the challenge of CNN optimization. Benchmarked against a reference implementation in TensorFlow, our Julia-based solution demonstrates notable advancements in training efficiency and performance. This work highlights Julia’s potential in deep learning, offering a streamlined tool for the deep learning community.

Index Terms—Automatic Differentiation, CNN, Julia Programming Language, Performance Optimization

I. INTRODUCTION

Deep learning has dramatically reshaped the technological landscape, offering groundbreaking solutions across various fields, from medical diagnostics to autonomous navigation. Convolutional Neural Networks (CNNs) have emerged as the linchpin for analyzing visual and sequential data, driving these advancements. The effectiveness of CNNs hinges on meticulous training and optimization processes, which heavily rely on automatic differentiation (AD). AD automates the computation of gradients, a cornerstone for the neural networks’ backpropagation algorithm.

Despite the proliferation of AD tools, the specialized nature of CNN computations requires bespoke libraries to maximize performance. The Julia programming language, recognized for melding the ease of use of dynamic languages with the efficiency of static ones, presents a fertile ground for such innovation. It promises to bridge the gap between high-level programming convenience and low-level computational efficiency, making it an apt choice for creating an AD library tailored for CNNs.

II. LITERATURE REVIEW

Julia is increasingly recognized for its applicability in scientific computing and machine learning due to its expressive syntax and computational speed. Bezanson et al. discuss these attributes, highlighting Julia as a high-level, high-performance programming language [1]. The language’s architecture offers a unique blend of ease of use and efficiency, making it an attractive choice for developing high-performance AD tools.

Baydin et al.’s survey [2] elucidates the foundational importance of AD in machine learning, offering insights into its mechanisms and applications. This underscores the necessity of efficient AD systems, particularly for the computationally demanding process of training CNNs.

Walther et al.’s exploration of targeted AD techniques [3] reveals the potential for optimizations specific to large-scale neural networks. Such targeted approaches are crucial for the effective training of CNNs, where computational resources are often a limiting factor.

Grøstad’s application of AD in Julia for solving PDEs [4] demonstrates Julia’s capability to handle complex mathematical operations efficiently, which is beneficial for implementing AD in the context of CNNs.

III. IMPLEMENTATION

The development of the library for computing convolutional neural networks began with deciding how to manage memory for the data needed to perform calculations. Efficient memory management is crucial for the performance and scalability of neural networks, particularly when dealing with large datasets and complex models. Two different approaches were considered: transition graphs and dynamic code generation for gradient computation. Transition graphs involve creating a static computation graph beforehand, while dynamic code generation involves generating the necessary computation code on-the-fly during runtime. After careful evaluation, the library was built using the dynamic code generation approach. This approach was selected because it allowed for greater flexibility and efficiency, particularly when paired with Julia language structures that support one-time memory allocation.

Four fundamental layers were selected for implementation in the library: convolution, max pool, fully connected, and flatten. Each of these layers serves a specific purpose in the architecture of a convolutional neural network and required a tailored implementation strategy. The convolution layer is responsible for feature extraction, the max pool layer reduces spatial dimensions, the fully connected layer serves as the classifier, and the flatten layer transforms the multidimensional output of the convolutional layers into a format suitable for the

fully connected layers. Each layer implementation needed to include three critical methods: forward pass, backward pass, and weight update. The forward pass computes the output of the layer, the backward pass computes the gradients needed for learning, and the weight update adjusts the parameters of the layer based on the computed gradients.

The entire implementation code is contained in the `src/` folder and in the files `JW_CNN.jl` and `utils.jl`. The `src/` folder includes the implementation of all layers, as well as an abstract version of the `Layer` struct. Each file is dedicated to the implementation of a specific layer and includes the structure storing the layer's data, a constructor, and the previously mentioned methods. Files `JW_CNN.jl` and `utils.jl` contain all the necessary elements for preparing the transition stack and organizing data clusters for each epoch, ensuring that the training process is efficient and systematic.

Additionally, to ensure the correctness of the implementation, a comprehensive set of tests was prepared and can be found in the `test/` folder. These tests are designed to validate the functionality of each layer and method, ensuring that the library performs as expected under various conditions. This rigorous testing framework is crucial for identifying and resolving any issues that may arise during development, ultimately leading to a more robust and reliable neural network library.

IV. CORECTNESS TESTS

Ensuring the correctness of the library is paramount, and this is primarily achieved through rigorous unit testing for each layer. Unit tests are designed to validate the functionality of individual components in isolation, allowing identification and repair of the issues at the earliest stage. Each layer in the library: convolution, max pool, fully connected, and flatten, has its own set of methods for testing the three key operations: the forward pass, the backward pass, and the weight update. Weight update tests are specifically prepared for the convolution and fully connected layers because weight updates do not occur in the max pool and flatten layers.

Forward pass tests

Forward pass tests are designed to verify that the layer computes the output correctly given a specific input. These tests involve preparing appropriate input data that mimics real-world scenarios, implementing the layer, and defining the expected output based on theoretical calculations or known benchmarks. The goal is to ensure that the layer produces the correct output consistently across a variety of input conditions.

Backward pass tests

The backward pass tests are slightly more complex as they not only validate the gradient calculations but also inherently test the forward pass. The backward pass computes the gradients needed for updating the weights during training. To test this, the forward pass is first executed to obtain the output, and then the backward pass is performed to compute the gradients. These tests serve as a form of integration test,

ensuring that both the forward and backward passes work harmoniously together. By validating the gradients, we ensure that the training process will adjust the weights correctly, leading to effective learning.

Weight update tests

Weight update tests are crucial for the convolution and fully connected layers. These tests ensure that the calculated gradients are applied correctly to update the weights. Proper weight updates are essential for the learning process, as they adjust the model parameters to minimize the loss function. By testing this aspect thoroughly, we ensure that the model improves its performance with each training iteration.

Benefits of testing

These comprehensive tests enabled the development of a correct and easily verifiable implementation. They facilitated modularity, allowing each layer to operate independently while collectively forming an integrated artificial neural network. This modular approach is beneficial for maintaining and extending the library, as changes or optimizations in one layer do not inadvertently affect others.

Role in optimization

This rigorous testing framework proved to be highly beneficial during the optimization phase. It provided a safety net, ensuring that any changes made to optimize performance did not alter the overall results. This allowed for more aggressive optimizations, because every unwanted behaviour will be detected. As a result, the library could be fine-tuned for better performance without compromising correctness, leading to a robust and efficient neural network implementation.

V. OPTIMIZATION

The most critical phase of developing the library involved optimizing computation and memory allocation. Below, I outline the key steps taken to achieve optimization and highlight which steps yielded the most significant results.

Logical Optimization

One of the most crucial and effective optimizations was the introduction of a conditional statement, which enabled further optimization efforts. This conditional statement was introduced in the backward pass method for the convolution layer. It checks whether the gradient with respect to the output is non-zero and, if so, proceeds with the computation; otherwise, it skips the calculation. This approach is vital as it reduces the number of operations in the backward pass for the convolution layer by approximately fourfold. The convolution layer's backward pass was the most time-consuming and memory-intensive among all the layers and passes.

Memory Allocation in the First Pass

Another essential aspect of optimization is memory allocation, which should occur only once in the library to conserve computational memory and time. During the first

pass, the code checks if the memory for a given structure has already been allocated. If not, memory allocation is performed. This ensures that in subsequent passes, memory allocation is replaced by variable assignment, which is faster than reallocation.

Macros @views and @inbounds

The macros @views and @inbounds significantly reduced memory allocation and execution time. The @views macro creates a reference to the original memory copy, avoiding unnecessary additional memory allocation, thus speeding up the program. The @inbounds macro disables bounds checking for array indices, which returns 0 if an index is out of bounds. Although this macro did not contribute substantially, it still sped up the program. The combination of these two macros shortened the epoch computation time and reduced memory allocation.

In-Place Computation

Using this method avoids creating new variables by operating on a single variable. The computations are performed sequentially and individually, eliminating the need for temporary variables. Instead, operations are carried out entirely on one variable, whose memory was allocated during the first pass. While this method is not universally applicable, it can be used in many instances.

Precomputing Loop Iterations

This method involves precomputing all possible loop states during the first pass to minimize computations inside the loop. An example is calculating the indices for subsequent passes in the max pool layer. Before performing calculations in the layer, all possible pass indices are stored in the layer's memory. As a result, the library retrieves indices from memory instead of recalculating them during layer computations.

Macro @threads

The final optimization involved enabling the library to operate on multiple threads using the @threads macro. Parallel programming is a common technique for accelerating program execution, though it comes with the critical limitation of memory access. Running the program on eight cores sped it up by approximately threefold. While this indicates that the algorithm is not highly optimized for parallel computation, the achieved speedup is satisfactory.

VI. COMPARISON

Comparison is the most crucial part of analyzing the effectiveness of the library. In this section, the implementation of a neural network using this library (referred to as JW_CNN) will be compared to implementations using Flux (also written in Julia) and TensorFlow (without an optimizer).

Implementation Assumptions:

- Input data: MNIST database
- Learning rate - 0.01

- Batch size - 100
- Threads - Julia environment set with 8 threads
- Layers
 - Convolution 3x3x6
 - Max Pool 2x2
 - Convolution 3x3x16
 - Max Pool 2x2
 - Flatten
 - Fully Connected from 400 to 84
 - Fully Connected from 84 to 10

Running this implementation in every library give this results:

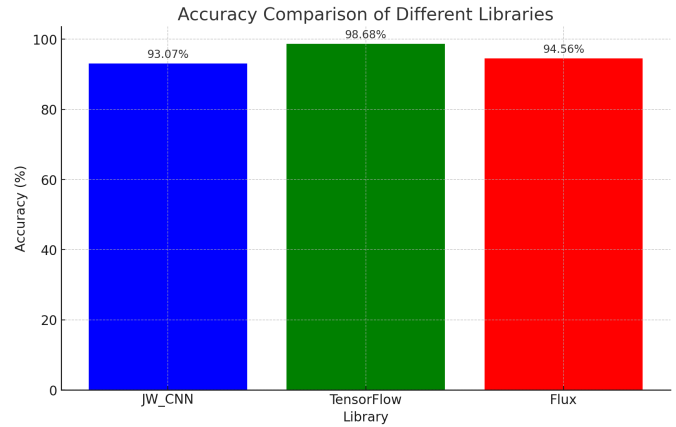


Fig. 1. Accuracy diagram

As shown, the best accuracy was achieved by the TensorFlow library, followed by Flux, and finally JW_CNN. The Python-based TensorFlow library performed the best in terms of accuracy, achieving almost exemplary results. On the other hand, the Flux implementation outperformed JW_CNN by only 1.5 percentage points, which can be considered a very good result and quite respectable accuracy.

In terms of speed, TensorFlow outperforms the other libraries, being three times faster than Flux and four times faster than JW_CNN. When comparing the libraries written in Julia, the difference in execution times is around ten seconds. Specifically, the execution time for one epoch in Flux is approximately 7 seconds, while in JW_CNN it is around 12 seconds. This indicates that if the number of epochs were increased, the difference in times would scale linearly. Excluding compilation time, it can be concluded that the Flux library is 1.73 times faster per epoch.

When examining memory allocation, the conclusions are not as straightforward. JW_CNN is significantly more memory-efficient, with memory usage being four times lower. This is an important aspect when training neural networks.

In summary, the JW_CNN library allows for training a network with slightly lower accuracy than the Flux library, which is written in the same language. The choice of the

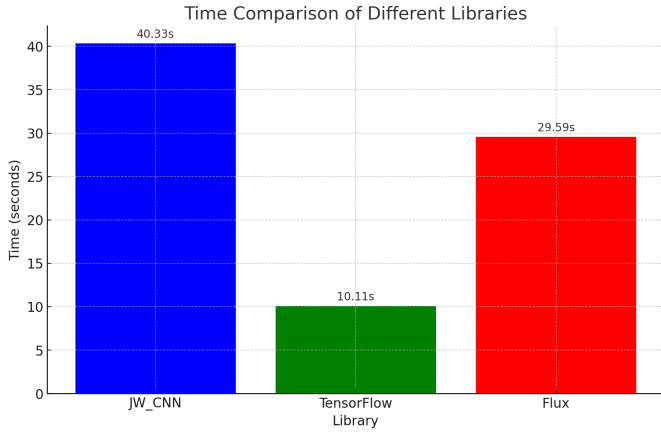


Fig. 2.

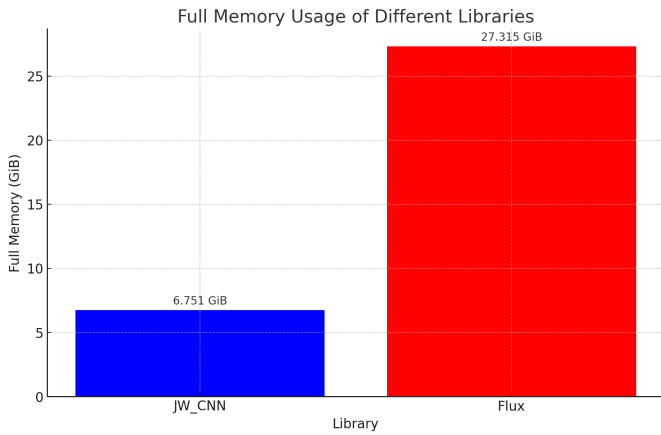


Fig. 3.

better library depends on the specific requirements. If memory efficiency is more critical, JW_CNN would be the better choice. However, if speed is the priority, Flux would be the preferred option. Including TensorFlow in the comparison highlights its superior performance, both in terms of accuracy and execution time, making it the best option among the three libraries evaluated.

VII. FINAL THOUGHTS

The development and optimization of the JW_CNN library for convolutional neural networks revealed valuable strategies for balancing accuracy, speed, and memory usage in machine learning applications. Key optimizations included introducing conditional logic in the backward pass of the convolution layer to reduce operations, allocating memory only once during the first pass to minimize computational overhead, and using macros like `@views` and `@inbounds` to reference original memory copies and disable bounds checking for array indices, respectively. In-place computation avoided the creation of temporary variables, and precomputing loop iterations reduced the computational burden during execution. Implementing multi-

threading with the `@threads` macro also demonstrated potential speed improvements through parallel processing.

When comparing JW_CNN with Flux and TensorFlow, TensorFlow stood out in terms of accuracy and speed, achieving near-perfect results and outperforming the other libraries significantly. Flux was slightly more accurate and 1.73 times faster than JW_CNN for a single epoch. However, JW_CNN excelled in memory efficiency, using four times less memory than its counterparts, which is critical for large-scale neural network training. Ultimately, the choice of library depends on the application's specific needs, whether it prioritizes speed, accuracy, or memory usage. The optimization techniques employed in JW_CNN underscore the importance of efficient memory management and computation in developing high-performance machine learning libraries.

The task proved to be highly demanding, requiring a significant amount of time and effort to develop the library. Countless hours were dedicated to researching, coding, and optimizing various aspects of the implementation. This involved not only the initial design and coding phases but also extensive testing and debugging to ensure the library's functionality and efficiency.

Despite the challenges, I am highly satisfied with the outcome. The implementation I have prepared is correct and demonstrates the capability to effectively train a neural network. Moreover, the library is quite well optimized, balancing the trade-offs between speed, memory usage, and accuracy. This project has been a significant learning experience, and I am proud of the progress and the final product.

REFERENCES

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, 'Julia: A Fresh Approach to Numerical Computing', *SIAM Rev.*, vol. 59, no. 1, pp. 65–98, Jan. 2017, doi: 10.1137/141000671.
- [2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, 'Automatic Differentiation in Machine Learning: a Survey'.
- [3] A. Walther et al., 'Getting Started with ADOL-C'. In: U. Naumann and O. Schenk, eds., *Combinatorial Scientific Computing*, Chapman-Hall CRC Computational Science, pp. 181–202, 2012.
- [4] S. Grøstad, 'Automatic Differentiation in Julia with Applications to Numerical Solution of PDEs', Master's thesis in Applied Physics and Mathematics, Norwegian University of Science and Technology Faculty of Information Technology and Electrical Engineering Department of Mathematical Sciences, 2019.